🏗️ คู่มือสร้างระบบ HR PRO ตั้งแต่ศูนย์

สอนสร้างระบบ HRIS ทีละขั้น — เขียนให้คนเพิ่งเริ่ม Laravel ก็ตามได้ อธิบายทั้ง **ทำอะไร · ทำยังไง · ทำไม**

ทุก Phase มี “พรอมต์สำเร็จรูป” ให้ก็อปไปสั่ง AI สร้างต่อได้

Laravel 12 | Blade ล้วน | RBAC เขียนเอง | ไม่มี Node build | MySQL / SQLite | Docker · Render

1 ภาพรวม & ปรัชญาการออกแบบ

HR PRO คือระบบบริหารทรัพยากรบุคคล (HRIS) สำหรับองค์กร 40–200 คน มีโมดูลครบ: พนักงาน, การลา, ลงเวลา, เงินเดือน, สรรหา, ประเมินผล, ฝึกอบรม, เอกสาร ฯลฯ

Stack ที่ใช้ และเหตุผล

ส่วน	เลือกใช้	ทำไม
ภาษา/เฟรมเวิร์ก	**PHP 8.2+ / Laravel 12**	ครบเครื่อง เอกสารดี deploy ง่าย
หน้าเว็บ (View)	**Blade ล้วน (Server-rendered)**	**ไม่ต้อง build ฝั่ง Node** — เขียนเสร็จรันได้เลย ลดความซับซ้อน
หน้าตา (CSS)	**CSS เขียนมือ** ใน `public/css/app.css`	ควบคุมได้ 100% ไม่พึ่ง Tailwind/Bootstrap
ฐานข้อมูล	**MySQL** (โปรดักชัน) / **SQLite** (เดโม/เทสต์)	สลับด้วยการแก้ `.env` บรรทัดเดียว
สิทธิ์ผู้ใช้	**RBAC เขียนเอง** (ไม่ใช้ Spatie)	เข้าใจกลไกเต็มที่ ตารางน้อย ปรับแต่งง่าย

หลักคิดสำคัญ 5 ข้อ

1. **ทุกโมดูลหน้าตาเหมือนกันหมด** — Controller + FormRequest + Service + Blade (index/create/edit) ทำโมดูลแรกเป็น โมดูลที่เหลือคือ “ก็อปแล้วเปลี่ยนชื่อ”
2. **Logic หนัก ๆ อยู่ใน Service ไม่ใช่ Controller** — Controller แค่รับ request → เรียก Service → ส่งไป View

3. **สิทธิ์เช็ค 2 ชั้น** — middleware กันที่ route + `@can` ซ่อน/โชว์ปุ่มใน View
4. **ทุกการกระทำสำคัญถูกบันทึก (Audit Log)**
5. **UI ประกอบจาก Blade Component** (`<x-button>` , `<x-card>` ...) เขียนครั้งเดียวใช้ซ้ำทั้งระบบ

โครงสร้างคำขอ 1 ครั้ง (Request lifecycle):

```
Browser → route (web.php)
        → middleware (auth → active → permission)
        → Controller (รับ input, ตรวจสิทธิ์ละเอียด)
        → FormRequest (validate)
        → Service (business logic)
        → Model/DB (Eloquent)
        → Blade View (+ Components) → HTML กลับไป Browser
```

2 เตรียมเครื่องมือ

- **PHP 8.2 ขึ้นไป** (`php -v`) พร้อม extension: `pdo_mysql` , `pdo_sqlite` , `mbstring` , `openssl` , `zip`
- **Composer** (`composer -V`)
- **MySQL 8** หรือใช้ SQLite ไปก่อน (ไม่ต้องติดตั้งอะไรเพิ่ม)
- **Git**

> 💡 **ไม่ต้องมี Node.js/npm**
> เพราะระบบนี้ไม่มีขั้นตอน build ฝั่งหน้าเว็บ

0 Phase 0 — ตั้งโครงโปรเจกต์

เป้าหมาย: ได้ Laravel เปล่า ๆ ที่เปิดในเบราว์เซอร์ได้

```bash
# 1) สร้างโปรเจกต์ Laravel ใหม่
composer create-project laravel/laravel hr-pro
cd hr-pro

# 2) ตั้งค่า .env แล้วสร้าง key
cp .env.example .env
php artisan key:generate

# 3) ลองเปิดเซิร์ฟเวอร์
php artisan serve      # http://localhost:8000
```

แก้ `.env` ส่วนฐานข้อมูล — เริ่มด้วย SQLite เพื่อความง่าย:

```dotenv
DB_CONNECTION=sqlite
# ลบ/คอมเมนต์ DB_HOST, DB_PORT, DB_DATABASE, DB_USERNAME, DB_PASSWORD
```

```bash
# สร้างไฟล์ฐานข้อมูลเปล่า
touch database/database.sqlite                    # macOS/Linux
New-Item database/database.sqlite -ItemType File   # Windows PowerShell
```

สร้าง `config/hrpro.php` เก็บค่าคงที่ของระบบ (เรียกด้วย `config('hrpro.xxx')`):

```php
<?php
return [
    'per_page' => 15,                          // จำนวนแถวต่อหน้าในตาราง
    'company_name' => env('HRPRO_COMPANY_NAME', 'HR PRO Co., Ltd.'),
    'work_start' => env('HRPRO_WORK_START', '09:00'),   // ใช้ตัดสินว่ามาสาย
    'work_end' => env('HRPRO_WORK_END', '18:00'),
    'late_grace_minutes' => env('HRPRO_LATE_GRACE_MINUTES', 15),
    'default_locale' => env('HRPRO_DEFAULT_LOCALE', 'th'),
];
```

1 Phase 1 — ฐานข้อมูล & Models

เป้าหมาย: ออกแบบตารางทั้งหมด + Model + ความสัมพันธ์ + Factory

1.1 ลำดับ Migration สำคัญมาก ⚠️

> ⚠️ **ตารางที่ถูกอ้างอิง (parent) ต้องสร้างก่อนตารางที่อ้างถึง (child)**
> เพราะมี foreign key — ตั้งชื่อไฟล์ migration ให้เรียงตามลำดับนี้

```
users → roles → permissions → permission_role → role_user
      → departments → positions → employees
      → leave_types → leave_requests → leave_balances
      → attendances → announcements → audit_logs → notifications
   (Phase 2) → payrolls, job_postings, candidates, performance_reviews,
               training_courses, training_enrollments, documents, onboarding_tasks
```

1.2 ตัวอย่าง migration (employees)

PHP

```php
Schema::create('employees', function (Blueprint $table) {
    $table->id();
    $table->foreignId('user_id')->nullable()->constrained()->nullOnDelete();
    $table->foreignId('department_id')->nullable()->constrained()->nullOnDelete();
    $table->foreignId('position_id')->nullable()->constrained()->nullOnDelete();
    $table->string('employee_code')->unique();
    $table->string('first_name');
    $table->string('last_name');
    $table->string('email')->unique();
    $table->string('status')->default('active');        // active | inactive |
terminated
    $table->string('employment_type')->default('full_time');
    $table->date('hire_date')->nullable();
    $table->decimal('base_salary', 12, 2)->nullable();
    $table->softDeletes();                              // ลบแบบกู้คืนได้
    $table->timestamps();
});
```

เคล็ดลับ

ใช้ `softDeletes()` กับข้อมูลคน/เอกสาร เพื่อให้ “ลบแล้วกู้คืนได้” ปลอดภัยกว่าลบจริง

1.3 ตัวอย่าง Model + ความสัมพันธ์ (Employee)

```php
class Employee extends Model
{
    use HasFactory, SoftDeletes;

    protected $fillable = [
        'user_id','department_id','position_id','employee_code',
        'first_name','last_name','email','status','employment_type',
        'hire_date','base_salary',
    ];

    protected function casts(): array
    {
        return ['hire_date' => 'date', 'base_salary' => 'decimal:2'];
    }

    // ----- ความสัมพันธ์ -----
    public function user()         { return $this->belongsTo(User::class); }
    public function department()   { return $this->belongsTo(Department::class); }
    public function position()     { return $this->belongsTo(Position::class); }
    public function leaveRequests(){ return $this->hasMany(LeaveRequest::class); }

    // ----- accessor สะดวก ๆ -----
    public function getFullNameAttribute(): string
    {
        return trim("{$this->first_name} {$this->last_name}");
    }
}
```

1.4 Factory (ข้อมูลปลอมไว้เทส/เดโม)

```php
public function definition(): array
{
    return [
        'employee_code' => 'EMP-'.fake()->unique()->numberBetween(100000, 999999),
        'first_name' => fake()->firstName(),
        'last_name' => fake()->lastName(),
        'email' => fake()->unique()->safeEmail(),
        'status' => 'active',
        'employment_type' => 'full_time',
        'hire_date' => fake()->dateTimeBetween('-5 years', 'now'),
        'base_salary' => fake()->numberBetween(18000, 90000),
    ];
}
```

```bash
php artisan migrate
```

Model ทั้งหมด (21 ตัว): User, Role, Permission, Department, Position, Employee, LeaveType, LeaveRequest, LeaveBalance, Attendance, Announcement, AuditLog, Notification, Payroll, JobPosting, Candidate, PerformanceReview, TrainingCourse, TrainingEnrollment, Document, OnboardingTask

2 Phase 2 — ระบบเข้าสู่ระบบ (Auth)

เป้าหมาย: หน้า login เอง (ไม่ใช้ scaffold สำเร็จรูป เพื่อควบคุมหน้าตา)

2.1 Controller

```php
class AuthenticatedSessionController extends Controller
{
    public function create() { return view('auth.login'); }

    public function store(Request $request): RedirectResponse
    {
        $credentials = $request->validate([
            'email' => ['required','email'],
            'password' => ['required'],
        ]);

        if (! Auth::attempt($credentials, $request->boolean('remember'))) {
            return back()->withErrors(['email' => 'อีเมลหรือรหัสผ่านไม่ถูกต้อง'])-
>onlyInput('email');
        }

        $request->session()->regenerate();          // กัน session fixation
        Auth::user()->update(['last_login_at' => now()]);
        return redirect()->intended(route('dashboard'));
    }

    public function destroy(Request $request): RedirectResponse
    {
        Auth::logout();
        $request->session()->invalidate();
        $request->session()->regenerateToken();
        return redirect()->route('login');
    }
}
```

2.2 Middleware “active” (กันบัญชีที่ถูกปิด)

```php
public function handle(Request $request, Closure $next): Response
{
    if (auth()->check() && ! auth()->user()->is_active) {
        auth()->logout();
        return redirect()->route('login')->withErrors(['email' => 'บัญชีนี้ถูกระงับการใช้งาน']);
    }
    return $next($request);
}
```

ลงทะเบียน alias ใน `bootstrap/app.php`:

```php
$middleware->alias([
    'permission' => \App\Http\Middleware\EnsureUserHasPermission::class,
    'active' => \App\Http\Middleware\EnsureUserIsActive::class,
]);
$middleware->redirectGuestsTo(fn () => route('login'));  // ยังไม่ล็อกอิน → หน้า login
```

3 Phase 3 — สิทธิ์การใช้งาน (RBAC)

เป้าหมาย: ระบบ “บทบาท (Role) → สิทธิ์ (Permission)” ที่ปรับแต่งได้ผ่านหน้าจอ

3.1 ตาราง (4 ตาราง)

```
roles            (id, slug, name, description, is_system)
permissions      (id, name, group, description)
permission_role  (permission_id, role_id)   ← M:N
role_user        (role_id, user_id)         ← M:N
```

3.2 รูปแบบชื่อสิทธิ์: `<ทรัพยากร>.<การกระทำ>`

เช่น `employees.view`, `employees.create`, `leave-approvals.approve`, `payroll.manage` — จัดกลุ่มด้วยคอลัมน์ `group`

3.3 เช็คสิทธิ์ใน Model + ผูก Gate

PHP — USER.PHP

```php
public function roles() { return $this->belongsToMany(Role::class)->withTimestamps(); }

public function hasPermission(string $permission): bool
{
    return $this->roles()
        ->whereHas('permissions', fn ($q) => $q->where('name', $permission))
        ->exists();
}
```

PHP — APPSERVICEPROVIDER::BOOT()

```php
// ทำให้ใช้ @can('employees.view') ใน Blade ได้
Gate::before(function (User $user, string $ability) {
    return $user->hasPermission($ability) ? true : null;
});
```

3.4 Middleware ตรวจสิทธิ์ระดับ route

PHP

```php
public function handle(Request $request, Closure $next, string $permission): Response
{
    abort_unless(auth()->user()?->hasPermission($permission), 403);
    return $next($request);
}
```

3.5 บทบาทมาตรฐาน 4 แบบ (seed ไว้ตั้งแต่ต้น)

บทบาท	slug	สิทธิ์
ผู้ดูแลระบบสูงสุด	`super-admin`	ทุกสิทธิ์ (`*`)
ผู้จัดการฝ่ายบุคคล	`hr-manager`	จัดการพนักงาน/ลา/ประกาศ/เงินเดือน...
หัวหน้างาน	`manager`	ดูทีม + อนุมัติการลา
พนักงาน	`employee`	บริการตนเอง (ลา/ลงเวลา/โปรไฟล์)

4 Phase 4 — Design System (หน้าตาเว็บ)

เป้าหมาย: วางหน้าตากลาง (layout + สี + component) ก่อนทำโมดูล เพื่อให้ทุกหน้าหน้าตาเหมือนกัน

4.1 Layout หลัก

BLADE — LAYOUTS/APP.BLADE.PHP

```
<!DOCTYPE html>
<html lang="th">
<head>
    <meta charset="utf-8">
    <meta name="csrf-token" content="{{ csrf_token() }}">
    <title>@yield('title','แดชบอร์ด') · {{ config('app.name') }}</title>
    <link href="...Inter + Noto Sans Thai..." rel="stylesheet">
    <link rel="stylesheet" href="{{ asset('css/app.css') }}">
</head>
<body>
  <div class="app">
    @include('partials.sidebar')      {{-- เมนูซ้าย --}}
    <div class="main">
      @include('partials.topbar')     {{-- แถบบน + ผู้ใช้ + แจ้งเตือน --}}
      <main class="content">
        @include('partials.flash')    {{-- ข้อความแจ้งผล --}}
        @yield('content')
      </main>
    </div>
  </div>
  <script src="{{ asset('js/app.js') }}" defer></script>
</body></html>
```

4.2 ตัวแปรสี (Design Tokens) ใน `public/css/app.css`

วาง “สีหลักของระบบ” เป็นตัวแปร CSS ที่เดียว แก้ทีเดียวเปลี่ยนทั้งเว็บ:

CSS

```
:root {
    --color-primary: #2563eb;     /* น้ำเงินหลัก (ปุ่ม/แถบหัวข้อ/ลิงก์) */
    --color-primary-dark: #1e3a5f;
    --color-bg: #f4f5f9;          /* พื้นหลังหน้า */
    --color-surface: #ffffff;     /* การ์ด/กล่อง */
    --color-text: #1f2937;
    --color-muted: #6b7280;
    --radius: 10px;
    --shadow: 0 1px 3px rgba(0,0,0,.08);
    --font: 'Inter','Noto Sans Thai',sans-serif;
}
body { background: var(--color-bg); color: var(--color-text); font-family: var(--font);
}
```

4.3 Blade Components (หัวใจของความเร็ว)

BLADE — COMPONENTS/BUTTON.BLADE.PHP

```
@props(['variant' => 'primary', 'type' => 'button'])
<button type="{{ $type }}" {{ $attributes->merge(['class' => "btn btn-$variant"]) }}>
    {{ $slot }}
</button>
```

BLADE — ใช้งานจริง

```
<x-button variant="primary">+ เพิ่มพนักงาน</x-button>
<x-stat label="พนักงานทั้งหมด" value="47" />
<x-card>...</x-card>
```

Component ทั้งชุด: alert, badge, button, card, checkbox, empty, icon, input, nav-link, page-header, select, stat, status-badge, textarea

4.4 Helper ฟังก์ชัน (`app/Support/helpers.php`)

```php
initials('สมชาย ใจดี');    // → "สใ"  (ตัวย่อทำ avatar)
thb(25000);                // → "฿25,000.00"
avatar_color('user@x');    // → สี hex คงที่จากชื่อ (avatar ไม่ต้องมีรูป)
```

```json
"autoload": { "files": ["app/Support/helpers.php"] }
```

5 Phase 5 — โมดูลหลัก (MVP) ทีละตัว

เป้าหมาย: ทำทีละโมดูล โดยทุกโมดูลใช้ pattern เดียวกัน 100%

🧩 **Pattern มาตรฐานของ 1 โมดูล (จำให้ขึ้นใจ)**

1. Migration + Model + Factory (ทำใน Phase 1 แล้ว)
2. Controller (resource: index/create/store/edit/update/destroy)
3. FormRequest (กฎ validation แยกจาก controller)
4. Service (ถ้ามี logic ซับซ้อน เช่น คำนวณวันลา/เงินเดือน)
5. Blade: index (ตาราง+ค้นหา+แบ่งหน้า), create, edit
6. ผูก route + permission middleware
7. บันทึก Audit Log เมื่อ create/update/delete

5.1 ตัวอย่างเต็ม: โมดูล “พนักงาน” (Employees)

(ก) Controller — กำหนดสิทธิ์ผ่าน `HasMiddleware`

PHP — EMPLOYEECONTROLLER.PHP

```
class EmployeeController extends Controller implements HasMiddleware
{
    public static function middleware(): array
    {
        return [
            new Middleware('permission:employees.view',   only: ['index','show']),
            new Middleware('permission:employees.create', only: ['create','store']),
            new Middleware('permission:employees.update', only: ['edit','update']),
            new Middleware('permission:employees.delete', only: ['destroy']),
        ];
    }

    public function index(Request $request): View
    {
        $employees = Employee::query()
            ->with(['department','position'])
            ->when($request->q, fn ($query, $q) =>
                $query->where(fn ($w) =>
                    $w->where('first_name','like',"%$q%")
                      ->orWhere('last_name','like',"%$q%")
                      ->orWhere('employee_code','like',"%$q%")))
            ->when($request->department, fn ($query, $d) => $query-
>where('department_id', $d))
            ->latest()
            ->paginate(config('hrpro.per_page'))
            ->withQueryString();      // คงค่าค้นหาไว้ตอนเปลี่ยนหน้า

        return view('employees.index', compact('employees'));
    }

    public function store(StoreEmployeeRequest $request, AuditLogger $audit):
RedirectResponse
    {
        $employee = Employee::create($request->validated());
        $audit->log('created', "เพิ่มพนักงาน: {$employee->full_name}");
        return redirect()->route('employees.index')->with('success', 'เพิ่มพนักงานเรียบร้อย');
    }
```

```
    // ... edit / update / destroy รูปแบบเดียวกัน
}
```

(ข) FormRequest — แยกกฎตรวจสอบ

PHP — STOREEMPLOYEEREQUEST.PHP

```php
public function rules(): array
{
    return [
        'first_name' => ['required','string','max:100'],
        'last_name' => ['required','string','max:100'],
        'email' => ['required','email','unique:employees,email'],
        'department_id' => ['nullable','exists:departments,id'],
        'base_salary' => ['nullable','numeric','min:0'],
    ];
}
```

(ค) View index — ใช้ component ที่สร้างไว้

BLADE — EMPLOYEES/INDEX.BLADE.PHP

```
@extends('layouts.app')
@section('title','พนักงาน')
@section('content')
    <x-page-header title="พนักงาน">
        @can('employees.create')
            <x-button onclick="location='{{ route('employees.create') }}'">+ เพิ่ม
พนักงาน</x-button>
        @endcan
    </x-page-header>

    <form method="GET" class="filters">
        <x-input name="q" placeholder="ค้นหา ชื่อ/รหัส/อีเมล" value="{{ request('q') }}" />
    </form>

    <x-card>
        <table class="table">
            <thead><tr><th>รหัส</th><th>ชื่อ</th><th>แผนก</th><th>สถานะ</th><th></th></tr>
</thead>
            <tbody>
            @forelse ($employees as $emp)
                <tr>
                    <td>{{ $emp->employee_code }}</td>
                    <td>{{ $emp->full_name }}</td>
                    <td>{{ $emp->department?->name ?? '-' }}</td>
                    <td><x-status-badge :status="$emp->status" /></td>
                    <td>@can('employees.update')<a href="{{
route('employees.edit',$emp) }}">แก้ไข</a>@endcan</td>
                </tr>
            @empty
                <tr><td colspan="5"><x-empty>ยังไม่มีข้อมูลพนักงาน</x-empty></td></tr>
            @endforelse
            </tbody>
        </table>
        {{ $employees->links() }}   {{-- แบ่งหน้าอัตโนมัติ --}}
    </x-card>
@endsection
```

(ง) Route

PHP — ROUTES/WEB.PHP (ในกลุ่ม MIDDLEWARE ['AUTH','ACTIVE'])

```
Route::resource('employees', EmployeeController::class);
```

5.2 รายการโมดูลหลัก MVP ทั้งหมด

โมดูล	สิทธิ์หลัก	Service / logic พิเศษ
พนักงาน (Employees)	`employees.*`	–
แผนก (Departments)	`departments.*`	หัวหน้าแผนก (manager_id)
ตำแหน่ง (Positions)	`positions.*`	ระดับ (level)
ประเภทการลา (Leave Types)	`leave-types.*`	โควต้าวัน/ปี
การลาของฉัน (My Leave)	พนักงานทุกคน	**LeaveService** — คำนวณวันลา หักโควต้า กันวันชนกัน
อนุมัติการลา (Approvals)	`leave-approvals.approve`	คืนโควต้าเมื่อปฏิเสธ/ยกเลิก
ลงเวลา (Attendance)	`attendance.viewAll`	**AttendanceService** — clock in/out, ตัดสินมาสาย
ประกาศ (Announcements)	`announcements.manage`	ปักหมุด (pinned)
แดชบอร์ด (Dashboard)	ทุกคน	**DashboardService** — รวมตัวเลขสรุป
รายงาน (Reports)	`reports.view`	สถิติรวม
ผู้ใช้ระบบ (Users)	`users.*`	ผูกบทบาท
บทบาท & สิทธิ์ (Roles)	`roles.*`	หน้าติ๊กสิทธิ์
บันทึกการใช้งาน (Audit Log)	`audit-logs.view`	อ่านอย่างเดียว
การแจ้งเตือน (Notifications)	ทุกคน	**NotificationService**
โปรไฟล์ (Profile)	ทุกคน	แก้ข้อมูล/เปลี่ยนรหัส

Service ทั้งหมด: AttendanceService, LeaveService, PayrollService, DashboardService, OrgChartService, NotificationService, AuditLogger

6 Phase 6 — โมดูลต่อยอด (Phase 2)

ทำหลัง MVP เสร็จ ใช้ pattern เดิมเป๊ะ แค่ logic ซับซ้อนขึ้น:

โมดูล	ไฮไลต์ที่ต้องทำ
เงินเดือน (Payroll)	`PayrollService::generateForPeriod($year,$month)` — คำนวณเงินเดือนทุกคนในงวด, รายการหัก/เพิ่ม
สรรหา (Recruitment)	JobPosting + Candidate (stage: applied→screening→interview→offer)
ประเมินผล (Performance)	รอบประเมิน, คะแนน, จุดแข็ง/จุดพัฒนา
ฝึกอบรม (Training)	TrainingCourse + TrainingEnrollment (ลงทะเบียน/สำเร็จ)
คลังเอกสาร (Documents)	อัปโหลดไฟล์ลง storage + สิทธิ์เข้าถึง
ผังองค์กร (Org Chart)	OrgChartService สร้างต้นไม้จาก department/manager
เริ่มงาน (Onboarding)	เช็กลิสต์งานต้อนรับ (OnboardingTask::TEMPLATE)
ส่งออก Excel/PDF	ExportController
แจ้งเตือน Email/LINE	Laravel Notification/Mail
สลับภาษา (i18n)	SetLocale middleware + LocaleController + ไฟล์แปลใน `lang/`

PHP — SETLOCALE.PHP

```php
public function handle(Request $request, Closure $next): Response
{
    app()->setLocale(session('locale', config('hrpro.default_locale')));
    return $next($request);
}
```

7 Phase 7 — ข้อมูลเริ่มต้น & Seeder

เป้าหมาย: ติดตั้งใหม่แล้วใช้งานได้ทันที + มีโหมดข้อมูลตัวอย่าง (เปิด/ปิดได้)

PHP — DATABASESEEDER.PHP

```php
public function run(): void
{
    // มีเสมอ: บทบาท/สิทธิ์ + admin 1 บัญชี (ไม่งั้น login ไม่ได้)
    $this->call([RolePermissionSeeder::class, AdminUserSeeder::class]);

    // ข้อมูลตัวอย่างเป็น "ทางเลือก" — เปิดด้วย env SEED_DEMO=true เท่านั้น
    if (filter_var(env('SEED_DEMO', false), FILTER_VALIDATE_BOOLEAN)) {
        $this->call(DemoDataSeeder::class);
    }
}
```

- **RolePermissionSeeder** — สร้างสิทธิ์/บทบาททั้งหมด (โครงสร้าง จำเป็นเสมอ)
- **AdminUserSeeder** — admin 1 บัญชีแบบ idempotent (`firstOrCreate`) ปรับผ่าน `ADMIN_EMAIL` / `ADMIN_PASSWORD`
- **DemoDataSeeder** — ข้อมูลปลอมทั้งหมด สร้างด้วย **Faker** ตอนรัน (ไม่เก็บในไฟล์)

BASH

```bash
php artisan migrate --seed                  # ติดตั้งสะอาด (roles + admin)
SEED_DEMO=true php artisan migrate --seed   # พร้อมข้อมูลตัวอย่าง
```

8 Phase 8 — การทดสอบ

ใช้ **PHPUnit** กับ SQLite in-memory (เร็ว ไม่กระทบ DB จริง) ตั้งใน `phpunit.xml` :

XML

```xml
<env name="DB_CONNECTION" value="sqlite"/>
<env name="DB_DATABASE" value=":memory:"/>
```

PHP — ตัวอย่างเทสต์ FEATURE

```php
public function test_hr_can_create_employee(): void
{
    $this->seed(RolePermissionSeeder::class);
    $hr = User::factory()->create();
    $hr->roles()->attach(Role::where('slug','hr-manager')->first());

    $this->actingAs($hr)
        ->post(route('employees.store'), [
            'first_name' => 'สมชาย', 'last_name' => 'ใจดี',
            'email' => 'somchai@test.local',
        ])
        ->assertRedirect();

    $this->assertDatabaseHas('employees', ['email' => 'somchai@test.local']);
}
```

BASH

```bash
php artisan test
./vendor/bin/pint        # จัดรูปแบบโค้ดตามมาตรฐาน Laravel
```

9 Phase 9 — นำขึ้นใช้งานจริง (Deploy)

Deploy เป็น **Docker container เดียว** ใช้ได้ทั้ง Render/Koyeb/Fly/Railway

SH — DOCKER/ENTRYPOINT.SH (แนวคิด)

```sh
[ -f .env ] || cp .env.example .env
[ -z "$APP_KEY" ] && php artisan key:generate --force
[ -f database/database.sqlite ] || touch database/database.sqlite

php artisan migrate --force
# seed ครั้งเดียวต่อ container; เปิดข้อมูลตัวอย่างด้วย SEED_DEMO=true
[ -f storage/.seeded ] || { php artisan db:seed --force && touch storage/.seeded; }

php artisan config:cache && php artisan route:cache && php artisan view:cache
exec php artisan serve --host=0.0.0.0 --port="${PORT:-8080}"
```

⚠️ **Free tier มี filesystem ชั่วคราว**
ข้อมูลถูก seed ใหม่ทุก redeploy เหมาะกับ “เดโม” ไม่ใช่ข้อมูลจริง — ถ้าต้องการถาวรให้ใช้ MySQL ภายนอกหรือ persistent disk

★ ชุดพรอมต์สำเร็จรูป

ถ้าอยากให้ AI (เช่น Claude Code) สร้างให้ ป้อนพรอมต์ทีละ Phase ตามนี้ (สั่งทีละขั้น อย่ารวบ):

Prompt 0 — ตั้งโครง

```
สร้างโปรเจกต์ Laravel 12 ชื่อ hr-pro สำหรับระบบ HRIS
ใช้ Blade ล้วน (ไม่มี Node build), CSS เขียนเองใน public/css/app.css,
ฐานข้อมูลเริ่มด้วย SQLite. สร้าง config/hrpro.php เก็บค่า per_page,
work_start, work_end, late_grace_minutes, company_name, default_locale.
อธิบายทุกขั้นและให้คำสั่งที่ต้องรัน.
```

Prompt 1 — ฐานข้อมูล

```
ออกแบบ migration + Eloquent model + factory สำหรับ HRIS โดยเรียงลำดับ
foreign key ให้ถูก: users, roles, permissions, permission_role, role_user,
departments, positions, employees, leave_types, leave_requests,
leave_balances, attendances, announcements, audit_logs, notifications.
employees ใช้ softDeletes. ใส่ relationship และ casts ให้ครบ.
```

Prompt 2 — Auth + RBAC

```
ทำระบบ login เอง (AuthenticatedSessionController) ไม่ใช้ scaffold,
และ RBAC เขียนเอง (ไม่ใช้ Spatie): ตาราง roles/permissions/permission_role/
role_user, เมธอด User::hasPermission(), ผูก Gate::before, middleware
'permission' และ 'active'. สร้าง RolePermissionSeeder ที่มีบทบาท
super-admin, hr-manager, manager, employee พร้อมสิทธิ์รูปแบบ
'<resource>.<action>'.
```

Prompt 3 — Design System

```
สร้าง layout app.blade.php (sidebar + topbar + content), public/css/app.css
ที่มี CSS variables (--color-primary: #2563eb ฯลฯ), ฟอนต์ Inter + Noto Sans Thai,
และ Blade components: alert, badge, button, card, checkbox, empty, icon, input,
nav-link, page-header, select, stat, status-badge, textarea. เพิ่ม helper
initials(), thb(), avatar_color() ใน app/Support/helpers.php.
```

Prompt 4 — โมดูล (ทำซ้ำต่อโมดูล)

```
สร้างโมดูล "<ชื่อโมดูล>" ตาม pattern มาตรฐาน: resource controller ที่ใช้
HasMiddleware ผูกสิทธิ์ <resource>.view/create/update/delete, FormRequest
แยก validation, (ถ้ามี logic ซับซ้อนให้แยกเป็น Service), Blade index
(ค้นหา+กรอง+แบ่งหน้าด้วย config('hrpro.per_page'))/create/edit ที่ใช้
component เดิม, route resource, และบันทึก Audit Log เมื่อ create/update/delete.
ทำโมดูล Employees ก่อนเป็นต้นแบบ.
```

Prompt 5 — Seeder แบบ opt-in

```
ปรับ DatabaseSeeder ให้ seed RolePermissionSeeder + AdminUserSeeder เสมอ
(admin แบบ idempotent ปรับผ่าน ADMIN_EMAIL/ADMIN_PASSWORD) และเรียก
DemoDataSeeder เฉพาะเมื่อ env SEED_DEMO=true. DemoDataSeeder ใช้ Faker
สร้างพนักงาน ~47 คน + การลา/ลงเวลา/ประกาศ/เงินเดือน.
```

Prompt 6 — Deploy

```
ทำ Dockerfile + docker/entrypoint.sh ที่ migrate, seed ครั้งเดียว (ตาม
SEED_DEMO), cache config/route/view แล้ว serve ที่ $PORT, พร้อม render.yaml
(Blueprint, autoDeploy, env APP_KEY/APP_URL/SEED_DEMO).
```

✓ สรุปลำดับการสร้าง (Checklist)

- [ ] Phase 0 — โครงโปรเจกต์ + .env + config/hrpro.php
- [ ] Phase 1 — migration + model + factory (เรียง FK ถูก)
- [ ] Phase 2 — login เอง + middleware active
- [ ] Phase 3 — RBAC + RolePermissionSeeder
- [ ] Phase 4 — layout + CSS tokens + Blade components + helpers
- [ ] Phase 5 — โมดูล MVP ทีละตัว (เริ่มที่ Employees)
- [ ] Phase 6 — โมดูล Phase 2 (Payroll, Recruitment, ...)
- [ ] Phase 7 — Seeder opt-in (admin + demo)
- [ ] Phase 8 — เขียนเทสต์ + pint
- [ ] Phase 9 — Docker + Render

เคล็ดลับสุดท้าย

อย่าทำทุกโมดูลพร้อมกัน — ทำ Employees ให้ครบวงจร (DB→Controller→View→Test) ก่อน 1 ตัว แล้วค่อยก็อปแพตเทิร์นไปโมดูลอื่น จะเร็วและพลาดน้อยที่สุด
